AGRUPAMIENTO Y PIVOTEO DE DATOS

M3: OBTENCIÓN Y PREPARACIÓN DE DATOS

|**AE6**: APLICAR TÉCNICAS DE UNIÓN, AGRUPAMIENTO Y PIVOTEO DE DATOS UTILIZANDO LA LIBRERÍA PANDAS PARA EL REACOMODO DE LA INFORMACIÓN PARA LA RESOLUCIÓN DE UN PROBLEMA.

alkemy

Introducción👩‍💻

El análisis de datos requiere técnicas avanzadas para organizar, resumir y estructurar la información de manera efectiva. El agrupamiento y el pivotado de datos son dos estrategias fundamentales que permiten reorganizar conjuntos de datos en formas más adecuadas para la exploración y el análisis.

El agrupamiento de datos consiste en segmentar un conjunto de datos en grupos basados en una o más variables, lo que facilita el análisis de tendencias, patrones y distribuciones. Por otro lado, el pivotado y despivotado de DataFrames permite reorganizar los datos cambiando su estructura, lo que resulta útil en análisis exploratorios y en la creación de reportes dinámicos.

Asimismo, la combinación de datos mediante técnicas como la concatenación y el uso de merge en Pandas es clave para integrar información de múltiples fuentes, asegurando que los conjuntos de datos puedan ser utilizados de manera eficiente en cualquier análisis.

Este manual proporciona una guía detallada sobre estas técnicas utilizando la biblioteca Pandas en Python. Se incluyen ejemplos prácticos y buenas prácticas para optimizar la manipulación de datos en diversos contextos analíticos.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

1. Comprender la importancia del **agrupamiento de datos** en el análisis de información.
2. Implementar **indexación jerárquica** para organizar datos de manera eficiente.
3. Utilizar **groupby** para segmentar datos y aplicar funciones agregadas.
4. Aplicar técnicas de **pivotado y despivotado** para reorganizar la estructura de los datos.
5. Combinar múltiples **DataFrames** mediante concatenación y merge en Pandas.
6. Optimizar el procesamiento de datos grandes mediante estas técnicas avanzadas.

AGRUPAMIENTO Y PIVOTEO DE DATOS

1. AGRUPAMIENTO CON INDEXACIÓN JERÁRQUICA

La indexación jerárquica en Pandas permite estructurar datos en múltiples niveles, facilitando su organización y segmentación. Es particularmente útil en conjuntos de datos con múltiples categorías relacionadas, ya que permite agrupar información sin necesidad de duplicar valores en columnas adicionales.

El uso de índices jerárquicos ayuda a representar datos de manera más compacta y estructurada, proporcionando mayor flexibilidad en la manipulación y el acceso a la información.

Ejemplo de creación de un índice jerárquico:

```
import pandas as pd

# Crear un DataFrame
datos = {'Región': ['Norte', 'Norte', 'Sur', 'Sur'],
         'Ciudad': ['A', 'B', 'A', 'B'],
         'Ventas': [100, 150, 200, 250]}

df = pd.DataFrame(datos)

# Definir un índice jerárquico
df.set_index(['Región', 'Ciudad'], inplace=True)
print(df)
```

Esto permite acceder a los datos de manera estructurada:

```
print(df.loc['Norte'])
```

Ventajas de la indexación jerárquica:

- Facilita la segmentación de datos en múltiples niveles.
- Permite acceder a subconjuntos específicos de manera eficiente.
- Mejora la organización y representación de datos en análisis tabulares.

- Reduce la redundancia en columnas, ya que se pueden almacenar múltiples niveles dentro del índice.

2. AGRUPAMIENTO DE DATOS CON LA FUNCIÓN GROUPBY

El método groupby() en Pandas permite dividir un conjunto de datos en grupos basados en una o más columnas y aplicar funciones agregadas a cada grupo. Este método es clave en la generación de estadísticas resumidas y en el análisis exploratorio de datos categorizados.

Ejemplo de uso de groupby() para calcular promedios:

```
# Crear un DataFrame
df = pd.DataFrame({'Departamento': ['Ventas', 'Ventas', 'TI', 'TI'],
                   'Empleado': ['Ana', 'Luis', 'Carlos', 'Sofía'],
                   'Salario': [50000, 54000, 60000, 62000]})

# Agrupar por Departamento y calcular el salario promedio
df_grouped = df.groupby('Departamento')['Salario'].mean()
print(df_grouped)
```

El uso de groupby() permite realizar análisis agregados sobre diferentes categorías en un conjunto de datos, facilitando la obtención de estadísticas clave.

Algunas de las funciones agregadas más comunes utilizadas con groupby() incluyen:

- .mean(): Media de los valores de cada grupo.
- .sum(): Suma total de los valores por grupo.
- .count(): Cantidad de elementos dentro de cada grupo.
- .max() y .min(): Valores máximo y mínimo dentro de cada grupo.
- .agg(): Aplicación de múltiples funciones agregadas a la vez.

Ejemplo de uso de .agg() para aplicar múltiples funciones:

```
# Aplicar múltiples funciones en el agrupamiento
df_grouped = df.groupby('Departamento').agg({'Salario': ['mean', 'max', 'min']})
print(df_grouped)
```

Este método es fundamental en el análisis de grandes volúmenes de datos y permite generar reportes con métricas clave para la toma de decisiones basada en datos.

3. PIVOTEO DE DATAFRAMES

El pivotado de datos convierte datos de formato largo a formato ancho, reorganizando los valores en nuevas columnas. Esta técnica permite transformar un conjunto de datos en el que los valores están organizados en filas en un formato más estructurado, facilitando su análisis y visualización.

El método pivot() de Pandas es una herramienta clave en este proceso. Se utiliza para reorganizar los datos en función de una o más columnas, creando una nueva estructura en la que los valores de una columna específica se convierten en encabezados de columnas.

Ejemplo de pivotado de datos:

```
import pandas as pd

# Crear un DataFrame
df = pd.DataFrame({'Fecha': ['2023-01', '2023-01', '2023-02', '2023-02'],
                   'Ciudad': ['A', 'B', 'A', 'B'],
                   'Ventas': [100, 150, 200, 250]})

# Pivotar el DataFrame
df_pivotado = df.pivot(index='Fecha', columns='Ciudad', values='Ventas')
print(df_pivotado)
```

Beneficios del pivotado de datos

- Permite visualizar la información en un formato tabular más intuitivo.
- Facilita la comparación de métricas a lo largo del tiempo o entre categorías.
- Es útil en la creación de reportes dinámicos y paneles de control.
- Optimiza la manipulación de datos en análisis exploratorios.

El pivotado es una herramienta poderosa en la estructuración de datos, pero requiere que el conjunto de datos no contenga valores duplicados en las columnas utilizadas para el índice y los encabezados.

4. DESPIVOTEO DE DATAFRAMES (MÉTODO MELT)

El proceso inverso al pivotado es el despivotado o melt, que transforma un DataFrame de formato ancho a formato largo. Esto es útil cuando se requiere normalizar datos para análisis o cuando un formato tabular no es adecuado para ciertas visualizaciones o modelos analíticos.

El método melt() en Pandas permite convertir un DataFrame en formato ancho a un formato largo, facilitando su análisis y manipulación.

Ejemplo de uso de melt():

```
# Despivotar el DataFrame
df_melt = df_pivotado.reset_index().melt(id_vars='Fecha', var_name='Ciudad',
                                         value_name='Ventas')
print(df_melt)
```

Ventajas del despivotado

- Facilita el manejo de datos cuando se trabaja con múltiples categorías.
- Permite compatibilizar los datos con herramientas de visualización y machine learning.
- Es útil en la transformación de datos para bases de datos relacionales.
- Mejora la flexibilidad de análisis al permitir realizar filtros y agregaciones dinámicas.

El despivotado es esencial en la preparación de datos para modelos predictivos y análisis en profundidad, asegurando que la información sea fácilmente manipulable.

COMBINACIÓN Y MERGE DE DATOS

5. CONCATENACIÓN DE DATAFRAMES

La concatenación de DataFrames permite unir múltiples conjuntos de datos a lo largo de filas o columnas. Es útil cuando se necesita agregar nuevos registros o combinar diferentes fuentes de datos con una estructura similar.

El método concat() en Pandas facilita la combinación de DataFrames, permitiendo especificar el eje de concatenación y manejar posibles valores duplicados.

Ejemplo de concatenación de DataFrames:

```
# Crear dos DataFrames
df1 = pd.DataFrame({'ID': [1, 2, 3], 'Valor': [100, 200, 300]})
df2 = pd.DataFrame({'ID': [4, 5, 6], 'Valor': [400, 500, 600]})

# Concatenar los DataFrames
df_concatenado = pd.concat([df1, df2])
print(df_concatenado)
```

Ventajas de la concatenación

- Permite combinar datos sin necesidad de relaciones explícitas.
- Es útil para unir conjuntos de datos de la misma estructura.
- Facilita la integración de múltiples fuentes de datos de manera rápida y sencilla.

La concatenación se utiliza frecuentemente en procesos de ETL (Extracción, Transformación y Carga) para consolidar datos antes de su análisis.

6. MERGE DE DATAFRAMES

El método merge() permite combinar dos DataFrames en base a una clave común, similar a una unión en bases de datos SQL. Es una técnica esencial cuando se trabaja con datos estructurados en múltiples tablas o fuentes de información.

Tipos de combinaciones (merge())

Existen diferentes tipos de combinaciones que pueden realizarse con merge() en Pandas:

- Inner join: Solo conserva los registros con coincidencias en ambas tablas.
- Left join: Mantiene todos los registros de la tabla izquierda y añade coincidencias de la tabla derecha.

- Right join: Conserva todos los registros de la tabla derecha y añade coincidencias de la tabla izquierda.
- Outer join: Mantiene todos los registros de ambas tablas, rellenando con valores nulos donde no haya coincidencias.

Ejemplo de uso de merge():

```
# Crear dos DataFrames
df1 = pd.DataFrame({'ID': [1, 2, 3], 'Nombre': ['Ana', 'Luis', 'Carlos']})
df2 = pd.DataFrame({'ID': [1, 2, 4], 'Salario': [50000, 54000, 60000]})

# Realizar un merge por la columna 'ID'
df_merged = pd.merge(df1, df2, on='ID', how='inner')
print(df_merged)
```

Beneficios del uso de merge()

- Facilita la integración de datos provenientes de distintas fuentes.
- Permite realizar análisis más completos al combinar información complementaria.
- Ayuda a estructurar datos en modelos relacionales sin redundancia.
- Optimiza la manipulación de grandes volúmenes de información en entornos analíticos.

El uso de merge() es fundamental en la consolidación de datos y en el preprocesamiento para modelos de machine learning, asegurando que toda la información relevante esté integrada en un solo DataFrame.

Cierre ✍️

El agrupamiento, pivotado y combinación de datos son técnicas fundamentales en el análisis y manipulación de datos estructurados. Estas herramientas permiten transformar conjuntos de datos en formatos que facilitan su exploración, interpretación y modelado, asegurando que la información esté organizada de manera óptima para la toma de decisiones.

El uso de la indexación jerárquica y groupby en Pandas facilita la agregación y segmentación de datos, proporcionando un marco flexible para analizar métricas en diferentes categorías. Por otro lado, el pivotado y despivotado de DataFrames permiten reorganizar la información en función de las necesidades del análisis, ya sea para generar reportes más claros o para preparar datos en estructuras normalizadas.

Asimismo, la capacidad de combinar DataFrames a través de concatenación y merge() es clave en la integración de múltiples fuentes de datos, permitiendo construir vistas unificadas de la información. Estas técnicas aseguran que el análisis se base en datos completos y consistentes, evitando duplicaciones o pérdidas de información relevante.

Dominar estas estrategias en Pandas es esencial para cualquier analista de datos, científico de datos o ingeniero de datos que trabaje con grandes volúmenes de información. Con una aplicación correcta, es posible optimizar flujos de trabajo, mejorar la calidad del análisis y garantizar que los datos sean interpretados de manera efectiva en distintos contextos empresariales y científicos.

Referencias📚

1. Pandas Documentation - https://pandas.pydata.org/docs/
2. Python for Data Analysis - Wes McKinney (2017).
3. Data Science from Scratch - Joel Grus (2019).
4. Machine Learning with Python - Andreas C. Müller, Sarah Guido (2017).
5. Hands-On Data Analysis with Pandas - Stefanie Molin (2019).
6. The Elements of Statistical Learning - Trevor Hastie, Robert Tibshirani, Jerome Friedman (2009).
7. An Introduction to Statistical Learning - Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani (2013).

¡Muchas gracias!

Nos vemos en la próxima lección

alkemy